



WEB Scrapping

LIBRERIAS DE ADQUISICIÓN DE DATOS. PARTE DEL
RA4

JAVIER ORTIZ LAGUNA

IES GASPAR MELCHOR DE JOVELLANOS

Contenido

1.	Introducción al Web Scraping en Python	2
2.	Herramientas principales en Python	2
3.	Instalación básica (lo más usado por principiantes)	2
4.	Ejemplo 1: Scraping simple con requests + BeautifulSoup	3
5.	Ejemplo 2: Cuando la página usa JavaScript (con Playwright)	4
6.	Pasos recomendados.....	4
7.	Mejores prácticas para un web scraping ético	5
8.	¿Qué es Cloudflare y por qué te complica la vida al hacer web scraping?	7
	Los “muros” que Cloudflare pone a los scrapers.....	7
	Cómo afecta en la práctica al scraping.....	8
	Soluciones más populares en 2025 (de más barata a más cara)	8
	Recomendación rápida según tu caso.....	9
9.	Otras webs para practicar web scraping de forma legal.....	9
10.	Evaluación del RA4.....	10

1. Introducción al Web Scraping en Python

El **web scraping** es la técnica de extraer información de páginas web de forma automática. Es muy útil para recopilar datos que no tienen API pública (precios, noticias, contactos, productos, etc.).

2. Herramientas principales en Python

Biblioteca	Uso principal	Ventajas	Desventajas
requests	Hacer peticiones HTTP (GET, POST, etc.)	Muy simple y rápida	No ejecuta JavaScript
BeautifulSoup	Parsear HTML/XML estático	Fácil de usar, excelente para HTML desordenado	Lento con páginas muy grandes
lxml	Parser más rápido que BeautifulSoup	Muy rápido	Sintaxis un poco más compleja
Selenium	Automatizar navegador real (Chrome, Firefox...)	Ejecuta JavaScript, ideal para SPA (React, etc.)	Mucho más lento y consume más recursos
Scrapy	Framework completo para proyectos grandes	Muy potente, asíncrono, pipelines, middlewares	Curva de aprendizaje más alta
playwright	Alternativa moderna a Selenium (más rápida)	Rápida, soporta múltiples navegadores	Más nueva, menos ejemplos antiguos

3. Instalación básica (lo más usado por principiantes)

```

```bash
>pip install requests beautifulsoup4 lxml
Opcional para páginas con JavaScript
>pip install selenium playwright
>playwright install # si usas playwright

```

...

## 4. Ejemplo 1: Scraping simple con requests + BeautifulSoup

```
```python
import requests
from bs4 import BeautifulSoup

# 1. Hacer la petición
url = "https://books.toscrape.com/"
response = requests.get(url)

# Verificar que todo fue bien
if response.status_code != 200:
    print("Error al obtener la página")
    exit()

# 2. Crear el objeto BeautifulSoup
soup = BeautifulSoup(response.text, 'html.parser')

# 3. Extraer datos (ejemplo: títulos y precios de libros)
libros = soup.find_all('article', class_='product_pod')

for libro in libros:
    titulo = libro.h3.a['title']
    precio = libro.find('p', class_='price_color').text
    print(f"{titulo} → {precio}")
...

```

5. Ejemplo 2: Cuando la página usa JavaScript (con Playwright)

```

```python
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
 browser = p.chromium.launch(headless=True) # headless=False
 para ver el navegador

 page = browser.new_page()
 page.goto("https://httpbin.org/html") # ejemplo simple
 # Esperar a que cargue algo específico
 page.wait_for_selector("h1")

 titulo = page.inner_text('h1')
 print("Título:", titulo)

 browser.close()
...

```

## 6. Pasos recomendados

1. Practica con sitios diseñados para aprender:

- <https://books.toscrape.com>
- <https://quotes.toscrape.com>
- <http://toscrape.com>

2. Aprende selectores CSS y XPath (muy útiles con ``.select()`` y ``.select_one()`` de BeautifulSoup).

3. Cuando domines lo básico → pasa a Scrapy o Playwright para proyectos más serios.

## 7. Mejores prácticas para un web scraping ético

El **web scraping ético** consiste en extraer datos de sitios web de manera responsable, respetando a los propietarios de los sitios, los usuarios y las leyes vigentes. Aunque el scraping de datos públicos no es ilegal en la mayoría de los casos (por ejemplo, en EE.UU. según fallos como hiQ vs. LinkedIn), puede violar términos de servicio, leyes de privacidad (como GDPR en Europa) o causar daños si no se hace correctamente.

Aquí tienes una lista de las **\*\*mejores prácticas\*\*** más recomendadas y actualizadas (basadas en guías de 2025):

### 1. Revisa y respeta el archivo robots.txt

Este archivo (en `https://ejemplo.com/robots.txt`) indica qué partes del sitio permiten crawlers. No es legalmente vinculante, pero ignorarlo es poco ético y puede llevar a bloqueos.

### 2. Lee los Términos de Servicio (ToS)

Muchos sitios prohíben explícitamente el scraping automatizado. Si lo violas, puedes enfrentar demandas por incumplimiento de contrato.

### 3. Prioriza APIs oficiales si existen

Es la forma más ética y legal. Sitios como Twitter/X, Reddit o Google ofrecen APIs para acceder a datos de manera controlada y respetuosa.

### 4. Limita la velocidad de las peticiones (rate limiting)

No sobrecargues el servidor: agrega delays (ej. 1-10 segundos entre requests), scrapea en horas de bajo tráfico. Usa `'time.sleep(1-3)'` o mejor aún, headers reales y rotación de IPs si es masivo.

### 5. Extrae solo datos públicos y necesarios

- Solo datos accesibles sin login.
- Evita información personal o sensible (PII: emails, direcciones, etc.), especialmente bajo GDPR/CCPA.
- No scrapees más de lo estrictamente needed (principio de minimización de datos).

## 6. Identifícate correctamente

Usa un User-Agent honesto (ej. incluye tu email o sitio web:

"MiScraper/1.0 (+miemail@dominio.com)". Evita spoofing o engaños.

Puedes usar headers reales para parecer un navegador:

```
```python
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0
Safari/537.36"
}

requests.get(url, headers=headers)
```
```

Pero mejor ser honesto para evitar represalias legales.

## 7. Maneja datos con responsabilidad

- Anonimiza si incluye info personal.
- No redistribuyas datos copyrighted sin permiso.
- Mantén logs de tu actividad para auditoría y transparencia.

## 8. Considera el impacto y sé transparente

Evalúa si tu scraping afecta el sitio (ej. consume muchos recursos). Si es a gran escala, contacta al propietario para pedir permiso o colaborar.

## 9. Cumple con leyes locales

- **\*\*Europa (GDPR)\*\***: Prohibido scrapear datos personales sin base legal.
- **\*\*EE.UU. (CFAA, CCPA)\*\***: Datos públicos suelen estar OK, pero evita accesos no autorizados.
- En general: Consulta un abogado para proyectos grandes o comerciales.

## 10. Usa herramientas éticas

Opta por frameworks como Scrapy (con middlewares para delays y respeto a robots.txt), o servicios que promueven prácticas responsables.

11. Sé respetuoso: no sobrecargues servidores pequeños.

Siguiendo estas prácticas, no solo evitas problemas legales, sino que contribuyes a un ecosistema web más saludable. El scraping ético es sostenible a largo plazo y genera confianza.

## 8. ¿Qué es Cloudflare y por qué te complica la vida al hacer web scraping?

**Cloudflare** es una empresa que protege sitios web contra ataques (DDoS, bots maliciosos, inyecciones SQL, etc.). Hoy en día protege más del **20 % de todo internet** (millones de sitios: desde tiendas online hasta blogs pequeños).

Cuando un sitio usa Cloudflare, todo el tráfico pasa por sus servidores antes de llegar al servidor real. Allí deciden si eres un humano legítimo o un bot/scraper.

### Los “muros” que Cloudflare pone a los scrapers

| Nivel de protección                | Qué ves tú como scraper                                       | Qué herramienta lo pasa fácil                                      | Qué herramienta suele fallar     |
|------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------|
| 1. Bot Management básico           | Nada especial, solo cabeceras normales                        | requests, httpx, Scrapy normal                                     | —                                |
| 2. “I’m under attack mode” / IUAM  | Página intermedia con “Checking your browser...” (5 segundos) | Playwright, Selenium, undetected-playwright, cloudscraper          | requests / httpx puro            |
| 3. JavaScript Challenge            | Te sirve un script JS que calcula un token (cf-clearance)     | Playwright, undetected-playwright, curl_cffi (a veces)             | requests, Scrapy sin integración |
| 4. Turnstile (nuevo CAPTCHA 2023+) | CAPTCHA invisible o checkbox “No soy un robot”                | undetected-playwright, servicios de solving (2Captcha, CapMonster) | Todo lo demás                    |



## WEB SCRAPING

| Nivel de protección                    | Qué ves tú como scraper                                | Qué herramienta lo pasa fácil                                          | Qué herramienta suele fallar       |
|----------------------------------------|--------------------------------------------------------|------------------------------------------------------------------------|------------------------------------|
| 5. Fingerprinting avanzado (2024-2025) | Comparan TLS fingerprint, Canvas, WebGL, fuentes, etc. | undetected-playwright, curl_cffi (versión 2025+), SeleniumBase UC Mode | Playwright normal, Selenium normal |
| 6. Bloqueo por comportamiento          | Detecta patrones de navegación no humana               | Navegadores reales con pausas, movimientos de ratón simulados          | Todo lo demás                      |

### Cómo afecta en la práctica al scraping

| Situación real                              | Qué pasa si no haces nada                                      | Solución más usada en 2025                                                          |
|---------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Tienda online pequeña con Cloudflare        | A veces pasa con requests, otras te da la página de desafío    | Probar primero con <b>curl_cffi</b> o <b>cloudscraper</b>                           |
| Gran ecommerce o sitio de tickets           | 100 % bloqueado con requests/Scrapy puro                       | Playwright o undetected-playwright                                                  |
| Sitio con Turnstile o fingerprinting fuerte | Incluso Playwright normal es detectado tras unas pocas páginas | <b>undetected-playwright</b> o servicios pagos (FlareSolvrr, ZenRows, ScrapeShield) |

### Soluciones más populares en 2025 (de más barata a más cara)

| Solución                                                    | Coste                | Dificultad | Éxito aproximado contra Cloudflare fuerte |
|-------------------------------------------------------------|----------------------|------------|-------------------------------------------|
| curl_cffi (nueva versión 2025)                              | Gratis               | Media      | 70-80 %                                   |
| cloudscraper                                                | Gratis               | Baja       | 40-60 % (solo desafíos antiguos)          |
| Playwright normal                                           | Gratis               | Baja       | 60-70 %                                   |
| undetected-playwright-python                                | Gratis               | Media      | 90-95 %                                   |
| Scrapy + scrapy-playwright                                  | Gratis               | Media-alta | 85-90 %                                   |
| FlareSolvrr (Docker auto-hosted)                            | Gratis (tu servidor) | Media      | 90-95 %                                   |
| Servicios pagos (BrightData, ZenRows, ScrapingBee, Oxylabs) | 30-500 €/mes         | Muy baja   | 98-99 %                                   |

| Solución                       | Coste                    | Dificultad | Éxito aproximado contra Cloudflare fuerte |
|--------------------------------|--------------------------|------------|-------------------------------------------|
| 2Captcha + cualquier navegador | ~1-3 € por 1000 CAPTCHAs | Media      | 95 %+ cuando hay Turnstile                |

## Recomendación rápida según tu caso

- Estás aprendiendo o proyecto pequeño → Empieza con **Playwright** normal.
- Te bloquean cada pocas peticiones → Cambia a **undetected-playwright**.
- Quieres seguir usando Scrapy puro → Instala **scrapy-playwright** o combina con **curl\_cffi**.
- Es un proyecto comercial importante → Considera servicios pagos (te ahorras meses de dolores de cabeza).

## 9. Otras webs para practicar web scraping de forma legal

Además de las mencionadas (books.toscrape.com, quotes.toscrape.com y toscrape.com), hay varios sitios diseñados específicamente como "sandboxes" para aprender web scraping sin preocuparte por bloqueos o violaciones de términos de servicio. Estos están pensados para principiantes y avanzados, y cubren técnicas como paginación, formularios, AJAX/JavaScript, iframes y más. Siempre verifica su robots.txt para confirmar, pero en general, son 100% permitidos.

Aquí una tabla con las mejores opciones recomendadas, basadas en recursos educativos actualizados:

| Sitio Web          | Descripción breve                                                               | Habilidades para practicar                                       | Nivel recomendado         |
|--------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------|
| scrapethissite.com | Plataforma con datasets reales (países, equipos de hockey, películas de Oscar). | Búsquedas, paginación, AJAX, iframes, manejo de errores comunes. | Principiante a intermedio |
| httpbin.org        | Sitio de pruebas para peticiones HTTP (responde con datos JSON simulados).      | Headers, cookies, sesiones, respuestas HTTP.                     | Principiante              |
| mockaroo.com       | Generador de datos falsos; puedes                                               | Extracción de tablas, datos estructurados.                       | Principiante              |

| Sitio Web                    | Descripción breve                                                                | Habilidades para practicar                         | Nivel recomendado     |
|------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------|-----------------------|
|                              | descargar o scrapear tablas de muestra.                                          |                                                    |                       |
| jsonplaceholder.typicode.com | API falsa con datos JSON (usuarios, posts, etc.), ideal para scraping ligero.    | Parsing JSON, peticiones GET/POST.                 | Principiante          |
| example.com                  | Página de ejemplo simple del IETF, con HTML básico y sin restricciones.          | Selectores CSS básicos, parsing HTML estático.     | Muy principiante      |
| wikipedia.org                | Enciclopedia abierta; permite scraping de datos públicos con límites razonables. | Infoboxes, tablas, categorías (revisa robots.txt). | Intermedio            |
| data.gov                     | Portal de datos abiertos del gobierno de EE.UU. (datasets públicos).             | Descarga y scraping de metadatos, APIs públicas.   | Intermedio a avanzado |

#### Consejos para practicar

- **Empieza con lo básico:** Usa scrapethissite.com para ejercicios guiados; tiene páginas listas con soluciones en varios lenguajes.
- **Ética siempre:** Aunque estos sitios lo permiten, no abuses (e.g., agrega delays entre requests). Para sitios reales como Wikipedia, limita a 1 request/segundo.
- **Herramientas:** Prueba con requests + BeautifulSoup para estáticos, o Selenium/Playwright para dinámicos.

Si quieres ejemplos de código para alguno de estos (e.g., scraping de scrapethissite), ¡avísame y lo armamos! ¿Cuál te interesa más?

## 10. Evaluación del RA4

Entrega los ejercicios contenidos al final de los archivos correspondientes a las librerías numpy, pandas, matplotlib y scikit-learn.

La entrega de estos ejercicios es individual.

Además, tendrás que realizar un ejercicio más grande a tu elección que implique la utilización de las 4 librerías anteriores más una de web-scraping de las descritas en este documento.

Consultar con el profesor el ejercicio que vais a realizar si no es uno de los ejemplos de este documento.

La entrega de este ejercicio se realizará en grupos de **3 o 4 personas**. Tendréis que realizar un documento en el que expliquéis el trabajo realizado, entregar el script o scripts Python realizados y, finalmente, tendréis que realizar una exposición/defensa del trabajo realizado.

A continuación, te muestro algunos ejemplos posibles:

#### 1. Análisis de Sentimiento en Reseñas de Películas (IMDB)

- **Scraping:** Extrae reseñas y calificaciones de películas de IMDB (ej. top 100). Usa BeautifulSoup para parsear páginas como <https://www.imdb.com/title/tt0111161/reviews> (cambia el ID por película). Recopila 200-500 reseñas con sus estrellas (1-10).
- **ML:** Convierte reseñas a vectores TF-IDF. Clasifica como positivo/negativo (umbral: >7 positivo). Entrena LogisticRegression. Evalúa accuracy en test set.

## 2. Predicción de Precios de Acciones (Yahoo Finance)

- **Scraping:** Scrapea datos históricos de acciones (ej. AAPL) de <https://finance.yahoo.com/quote/AAPL/history>. Extrae fecha, open, close, volume para los últimos 6 meses.
- **ML:** Usa características como open/volume para predecir close. Divide en train/test. Entrena LinearRegression o RandomForestRegressor. Mide MSE.

## 3. Clasificación de Noticias por Categoría (BBC News)

- **Scraping:** Navega por <https://www.bbc.com/news> y extrae títulos/descripciones de artículos en secciones como "World", "Business". Recopila 100 por categoría.
- **ML:** Vectoriza textos con TF-IDF. Etiqueta por categoría. Entrena RandomForestClassifier. Calcula accuracy y confusion matrix.

## 4. Predicción de Precios de Viviendas (Zillow o Similar)

- **Scraping:** Scrapea listados de casas de un sitio como Craigslist (ej. housing section). Extrae precio, dormitorios, ubicación, metros cuadrados (limita a una ciudad).
- **ML:** Codifica categóricas (ej. ubicación con OneHot). Predice precio con LinearRegression. Evalúa con RMSE en test set.

## 5. Análisis de Sentimiento en Tweets (Scraping de X/Twitter vía Archivo Público)

- **Scraping:** Usa BeautifulSoup en páginas archivadas o un scraper simple para hashtags (nota: Twitter restringe, usa datasets públicos o APIs). Extrae textos de posts sobre un tema (ej. #AI).
- **ML:** Clasifica sentimiento (positivo/negativo) usando NaiveBayesClassifier de scikit-learn. Vectoriza con CountVectorizer. Mide precisión.

## 6. Clasificación de Productos en E-commerce (Amazon)

- **Scraping:** Extrae nombres, descripciones y categorías de productos de páginas de búsqueda en Amazon (ej. "laptops"). Limita a 200 items para evitar bans.
- **ML:** Usa descripciones para predecir categoría (ej. electrónica vs. libros). Entrena SVC con TF-IDF. Evalúa con cross-validation.

## 7. Predicción de Temperatura con Datos Meteorológicos (Weather.com)

- **Scraping:** Scrapea datos diarios de clima de una ciudad en [https://weather.com/weather/monthly/l/\[ciudad\]](https://weather.com/weather/monthly/l/[ciudad]). Extrae temp max/min, precipitación para meses pasados.
- **ML:** Usa fechas y precipitación para predecir temp max. Entrena RandomForestRegressor. Visualiza predicciones vs. reales.

## 8. Detección de Spam en Comentarios (Reddit)

- **Scraping:** Extrae comentarios de subreddits (ej. r/technology) usando BeautifulSoup en páginas como <https://www.reddit.com/r/technology/comments>. Etiqueta manualmente algunos como spam/no-spam.
- **ML:** Vectoriza textos. Entrena LogisticRegression para clasificar spam. Usa accuracy y recall.

## 9. Recomendación de Libros por Género (Goodreads)

- **Scraping:** Scrapea títulos, autores, ratings y géneros de listas en <https://www.goodreads.com/list>. Recopila 300 libros.
- **ML:** Clusteriza libros por similitud (TF-IDF en descripciones) con KMeans. O clasifica género con DecisionTreeClassifier.

## 10. Predicción de Salarios en Ofertas de Empleo (Indeed)

- **Scraping:** Busca jobs en <https://www.indeed.com/jobs?q=data+scientist> y extrae título, salario estimado, experiencia requerida.
- **ML:** Codifica experiencia (ej. junior/senior). Predice salario con LinearRegression. Evalúa MSE y ajusta hiperparámetros con GridSearchCV.

Los algoritmos de ML indicados, así como la evaluación son orientativos. Podéis utilizar otros algoritmos y otras formas de evaluar los modelos a vuestra elección pero que sean adecuados con lo que estáis haciendo.

A continuación, se muestran algunos de los ejercicios ampliados:

## Ejercicios Prácticos más descriptivos

### Ejercicio 2: Predicción de Precios de Acciones (Yahoo Finance)

**Enunciado:** Desarrolla un script en Python para predecir precios de cierre de acciones usando datos scrapeados de Yahoo Finance. Este ejercicio combina web scraping de datos financieros, procesamiento temporal y modelado predictivo para entender tendencias bursátiles.

- **Scraping:** Utiliza la biblioteca Requests para obtener el HTML de la página <https://finance.yahoo.com/quote/AAPL/history> (o similar para otra acción como GOOGL). Emplea BeautifulSoup para parsear la tabla de datos históricos y extrae al menos las columnas de fecha, open (apertura), close (cierre) y volume (volumen) para los últimos 6 meses (aproximadamente 120-150 filas, ajustando parámetros de URL como `?period1=fecha_inicio&period2=fecha_fin` para filtrar fechas). Incluye headers en tus requests (ej. User-Agent) para evitar bloqueos y respeta el rate limiting (pausas con `time.sleep`). Maneja posibles cambios en la estructura HTML inspeccionando elementos como

y .

- **Procesamiento:** Convierte los datos extraídos en un DataFrame de Pandas. Usa NumPy para calcular estadísticas como medias móviles (rolling mean) de close y volume, y maneja valores nulos o outliers (ej. interpolación con np.interpolate). Agrega features derivadas, como el cambio porcentual diario (close - open) / open.
- **ML:** Divide los datos en train/test (80/20) usando train\_test\_split de Scikit-learn. Usa características como open, volume y medias móviles para predecir close. Entrena múltiples modelos: LinearRegression, DecisionTreeRegressor, RandomForestRegressor (con n\_estimators=100). Evalúa con métricas como MSE, MAE y  $R^2$  (usando mean\_squared\_error, mean\_absolute\_error, r2\_score). Ajusta hiperparámetros con GridSearchCV para el mejor modelo.
- **Visualización:** Grafica las predicciones vs. valores reales con Matplotlib (línea para reales, puntos para predicciones), incluyendo bandas de error (std dev con NumPy). Muestra un scatter plot de open vs. close coloreado por volume.

**Consideraciones:** Yahoo Finance permite scraping limitado; si hay CAPTCHA, considera usar la API de yfinance como alternativa ética. Entregable: Código fuente (con comentarios), CSV de datos scrapeados, reporte con métricas (tabla Pandas), gráficos guardados como PNG y modelo serializado con Joblib.

#### Ejercicio 4: Predicción de Precios de Viviendas (Zillow o Similar)

**Enunciado:** Crea un programa para predecir precios de viviendas basado en datos inmobiliarios scrapeados, integrando codificación de variables y evaluación de modelos para simular un análisis de mercado real.

- **Scraping:** Emplea Scrapy (o Requests + BeautifulSoup) para scrapear listados de casas de un sitio como Idealista.es (ej. URL base [https://www.idealista.com/venta-viviendas/\[ciudad\]/](https://www.idealista.com/venta-viviendas/[ciudad]/)). Limita a una ciudad específica (ej. Madrid) y extrae al menos 100 entradas con precio, número de dormitorios, ubicación (barrio o código postal), y metros cuadrados. Navega por páginas de resultados (pagination) usando XPath o CSS selectors para elementos como

. Incluye proxies o headers rotativos si es necesario, y limita requests para cumplir con términos de servicio.

- **Procesamiento:** Organiza los datos en un DataFrame de Pandas, limpia strings (ej. eliminar símbolos de moneda con str.replace) y convierte a tipos numéricos. Usa NumPy para imputar valores faltantes (ej. media con np.nanmean) y normalizar features como metros cuadrados (escalado min-max).
- **ML:** Codifica variables categóricas (ej. ubicación con OneHotEncoder de Scikit-learn). Divide en train/test. Predice precio usando modelos como LinearRegression, DecisionTreeRegressor y GradientBoostingRegressor. Evalúa con métricas como MSE, RMSE ( $\text{np.sqrt}(\text{MSE})$ ) y  $R^2$ . Compara modelos en una tabla Pandas.

- **Visualización:** Usa Matplotlib para un boxplot de precios por dormitorios, un heatmap de correlaciones (Pandas corr() + Matplotlib imshow), y un scatter de predicciones vs. reales.

**Consideraciones:** Sitios como Idealista prohíben scraping masivo; usa para fines educativos y considera APIs alternativas como la de Zillow si disponible. Entregable: Spider de Scrapy (o script simple), DataFrame exportado a CSV, métricas en tabla, gráficos PNG y modelo entrenado.

### Ejercicio 5: Análisis de Sentimiento en Tweets (Scraping de X/Twitter vía Archivo Público)

**Enunciado:** Implementa un sistema para analizar sentimiento en posts de X (Twitter) sobre un tema actual, usando scraping ético o datasets públicos, y clasificación de ML para identificar opiniones.

- **Scraping:** Dado las restricciones de X en 2025 (API paga y anti-scraping), usa BeautifulSoup en páginas archivadas (ej. vía Wayback Machine) o un scraper simple para hashtags públicos (nota: evita scraping directo; prefiere datasets como los de Kaggle o APIs gratuitas como Tweepy con credenciales). Extrae textos de al menos 200 posts sobre un tema (ej. #AI), incluyendo fecha y autor si disponible. Si usas scraping, parsea elementos como

en URLs como [https://twitter.com/search?q=%23AI&src=typed\\_query](https://twitter.com/search?q=%23AI&src=typed_query) (con Selenium si dinámico).

- **Procesamiento:** Crea un DataFrame en Pandas con los textos. Preprocesa con NumPy y NLTK (tokenización, remoción de stop words). Vectoriza textos con CountVectorizer o TfidfVectorizer de Scikit-learn, y usa NumPy para manejar matrices sparse.
- **ML:** Etiqueta un subconjunto manualmente o usa un dataset pre-etiquetado para entrenamiento. Clasifica sentimiento (positivo/negativo/neutro) usando modelos como GaussianNB (Naive Bayes), LogisticRegression, DecisionTreeClassifier y RandomForestClassifier. Evalúa con accuracy, precision, recall, F1-score (classification\_report) y matriz de confusión.
- **Visualización:** Grafica la distribución de sentimientos con un pie chart en Matplotlib, y la matriz de confusión como heatmap.

**Consideraciones:** Respeta la privacidad; usa datos anónimos y públicos. Si no hay acceso, descarga un dataset CSV de Hugging Face. Entregable: Script de scraping/alternativa, DataFrame procesado, reporte de métricas (tabla), gráficos y modelo guardado.

### Ejercicio 7: Predicción de Temperatura con Datos Meteorológicos (Weather.com)

**Enunciado:** Desarrolla un predictor de temperaturas usando datos climáticos scrapeados, enfocándote en series temporales para pronosticar patrones meteorológicos.



- **Scraping:** Usa Requests y BeautifulSoup para scrapear datos diarios de clima de una ciudad (ej. New York) en [https://weather.com/weather/monthly/l/\[ciudad\]](https://weather.com/weather/monthly/l/[ciudad]) (reemplaza [ciudad] por código como USNY0996). Extrae temp max/min, precipitación y fechas para al menos 12 meses pasados (navega por meses anteriores). Parsea la tabla calendario con selectores como

y maneja formatos de fecha.

- **Procesamiento:** Convierte a DataFrame Pandas con índice datetime. Usa NumPy para smoothing (ej. filtros moving average con `np.convolve`) y manejo de arrays temporales. Agrega features como día del año (`np.sin` para ciclicidad).
- **ML:** Usa fechas (convertidas a ordinales) y precipitación para predecir temp max. Entrena modelos como LinearRegression o ARIMA (integrable con statsmodels, pero usa Scikit-learn para base). Divide en train/test cronológico. Evalúa con MAE y RMSE.
- **Visualización:** Grafica series temporales de predicciones vs. reales con Matplotlib (líneas), incluyendo intervalos de confianza (std con NumPy). Agrega un scatter de precipitación vs. temp.

**Consideraciones:** Weather.com limita scraping; usa con moderación o APIs como OpenWeatherMap. Entregable: Código scraping, CSV de datos, métricas, gráficos PNG y visualización interactiva opcional.

### Ejercicio 10: Predicción de Salarios en Ofertas de Empleo (Indeed o InfoJobs)

**Enunciado:** Crea un modelo para predecir salarios en ofertas de empleo scrapeadas, incorporando optimización de hiperparámetros para mejorar precisión en análisis laboral.

- **Scraping:** Busca jobs en <https://www.indeed.com/jobs?q=data+scientist> (o InfoJobs.es similar) y usa BeautifulSoup para extraer al menos 150 entradas con título, salario estimado (si disponible), y experiencia requerida (parsea

). Maneja paginación (`?start=10`) y extrae de múltiples páginas.

- **Procesamiento:** DataFrame Pandas para datos. Usa NumPy para parsear salarios (ej. promedio de rangos con `np.mean`) y codificar experiencia (mapear strings a numéricos). Imputa faltantes con mediana.
- **ML:** Codifica experiencia (ej. OrdinalEncoder para junior/mid/senior). Predice salario con LinearRegression como base, pero prueba DecisionTreeRegressor. Evalúa con MSE y  $R^2$ . Ajusta hiperparámetros (ej. `max_depth` para árboles) usando GridSearchCV de Scikit-learn.
- **Visualización:** Bar plot de salarios promedio por experiencia con Matplotlib, y scatter de predicciones vs. reales.

**Consideraciones:** Indeed prohíbe scraping automatizado; usa para educación o APIs. Limita a datos públicos. Entregable: Script, CSV, tabla de métricas (con GridSearch results), gráficos y modelo optimizado.